RouteVoice: A Hardwired-Codebook Speech Recognizer in Open 180 nm Silicon, Built with AI Coding Agents

Sanjay Prasad H S

Abstract

We present RouteVoice, an on-device speech recognizer whose transformer weights are hardwired into silicon. Each weight is a fixed 4-bit index into a 16-entry codebook shared by 64 weights. Every multiplication becomes two selections from a table of partial products computed once per input activation and shared by all 320 output lanes.

Applied to the 34 M-parameter Moonshine Tiny ASR model, this format stays within +2 word-error-rate points of the original on a reserved test. It also reduces on-chip weight storage from 28.6 MB to 17.5 MB. The full recognizer RTL reproduces the frozen model's tokens and intermediate hashes bit-exactly from index+codebook storage.

We implemented the largest projection (1280 × 320, 409,600 MACs) in the open GF180MCU 180 nm process with an all-open-source flow. It is routed with zero routing, Magic and KLayout DRC violations and zero antenna violations, matches its netlist under LVS, and meets timing at 150 ns in all nine PVT/RC corners after two RTL pipelining changes. Measured from a real-speech activity trace, routed power is 0.188 W, or 287 pJ per MAC. Mask-programmed weight banks for all 98 matrices occupy 174.7 mm^2; all are DRC-clean and read back against the model, with full LVS on representative banks.

The work was carried out by AI coding agents under human direction, at a cloud cost of $34.82. We report the procedure, its failures, and practices that would shorten it. We also analyse what hardwired codebook silicon implies for local language models and for multi-purpose transformer chips.

1 Introduction

Most AI inference still runs in data centres. Voice interfaces for appliances, wearables, toys and sensors face a practical problem: the device must either send audio to the cloud, with cost, latency and privacy consequences, or run a model locally. Running locally usually means a general-purpose NPU or microcontroller with external memory, so weights are fetched from memory on every use.

For a device whose model never changes, weight movement is pure overhead. A model that is fixed for the life of a product can instead be built into the silicon.

Hardwired, or "model-specific", inference silicon has recently attracted commercial attention at the frontier, where fixed-weight chips promise very high token rates for large language models [5]. This paper asks the complementary question at the other end of the scale:

> *Can a useful speech model be hardwired into an open, mature, low-cost process, and can a very small team build and verify it using AI coding agents and open-source tools?*

We present RouteVoice, a Moonshine Tiny speech recognizer [25] designed for GlobalFoundries' open 180 nm process (GF180MCU). Every weight is a fixed 4-bit index into a small local codebook. Multiplications become selections from a table of partial products shared across all output lanes.

The building blocks are not new:

- codebook-indexed weights [19, 67];
- shared multiples of an input selected per fixed coefficient [43], which hardwired-LLM patents also describe [7, 4];
- hardwired weights on an open PDK [40].

What we add is their combination in a complete, verified speech recognizer, carried through an open physical flow and documented end to end.

Contributions.

1. **A hardwired-codebook weight format, validated for ASR.** K64 stores each weight as a 4-bit index into a 16-entry UINT8 book shared by 64 weights. It keeps Moonshine Tiny within +2 WER points of the original on a reserved test, and cuts on-chip weight storage by 39%.
2. **A bit-exact recognizer.** The full recognizer RTL decodes speech to the exact tokens and intermediate hashes of the frozen software model, with every weight served from index+codebook storage.
3. **A routed, signoff-checked projection engine.** The 1280 × 320 engine (409,600 MACs per projection) closes timing at 150 ns in all nine PVT/RC corners. Routing, Magic DRC, KLayout DRC, antenna and LVS checks are clean. Activity-annotated routed power is 287 pJ/MAC.
4. **Full-design evidence.** Whole-recognizer synthesis for silicon and FPGA completes, and DRC-clean mask-programmed weight banks cover all 98 matri-

ces (174.7 mm^2), with full LVS on six representative banks.

5. **A documented agent-driven methodology.** We report the step-by-step procedure, the failures and their fixes, and the cost ($34.82 of cloud compute plus a laptop), and derive concrete practices for faster flows.
6. **An analysis of future scope.** We size what the same approach implies for local LLMs, and propose ways to make hardwired silicon serve multiple tasks.

Section 2 reviews related work. Sections 3 and 4 describe the weight format and architecture, and section 5 the methodology. Sections 6 and 7 report results, including a timing-closure case study. Sections 8 to 10 discuss efficiency, future scope and limitations.

2 Background and Related Work

Hardwired and model-specific inference silicon. Fixing a network's coefficients in hardware is old: bespoke printed classifiers hardwire model coefficients and cut area and power by an order of magnitude [39]. ROM-based compute-in-memory keeps weights in dense mask ROM and adds a small SRAM path for adaptation:

- YOLoC [11] and Hidden-ROM [10] do this for CNNs;
- ROMA [61] puts a quantized LLM base model in ROM and keeps LoRA weights and the key/value cache in SRAM.

At the frontier, Taalas has announced HC1, which builds Llama 3.1 8B into a single die and reports about 17k tokens/s per user [5, 68]; these are vendor figures. Its patent applications describe two mechanisms closely related to ours:

- one multiplier per possible weight value, with a mask-configured mesh routing products to accumulators [7];
- weights stored as hardware pointers into small codebooks [4], alongside a shared-connection mask ROM [6].

HNLPU [35] studies the economics of hardwiring a 120 B-parameter model at 5 nm, embedding weights in metal topology to share most mask layers across chips. Etched's transformer-specific ASIC [38] specialises to an architecture class rather than to one set of weights.

Closest to our setting, Ankhdjet [40] compiles BitNet b1.58 ternary weights into the via mask of a compute-in-ROM macro on SKY130 and signs it off with open tools. Our work is therefore *not* the first hardwired-weight design on an open PDK. It differs in scope: a complete encoder–decoder speech recognizer, rather than one matrix macro, verified from audio to tokens. It also differs in mechanism: 4-bit codebook indices selecting shared radix-16 partial products, rather than ternary ROM sensing.

Codebook quantization and shared products. Storing 4-bit indices into a 16-entry table goes back to Deep Compression [20] and its accelerator EIE [19]. Mokey [67] applies 4-bit dictionaries to transformers. Modern LLM quantizers explore richer codebooks: AQLM [15], QuIP# [55], VPTQ [36] and GPTVQ [57] use vector codebooks, and SqueezeLLM [27] uses non-uniform scalar look-up tables. Our K64 format is a scalar non-uniform quantizer of this last kind.

Forming multiples of an input once and selecting them per fixed coefficient is also established:

- **DSP:** computation-sharing multipliers [43], multiplierless multiple-constant multiplication [59] and distributed arithmetic [65];
- **software:** LUT-GEMM [42] and T-MAC [64] apply the same idea to quantized LLMs on GPUs and CPUs;
- **run-time weight reuse:** UCNN [23] groups computation by repeated weight values while the network runs.

Our datapath combines these known pieces: per-group 16-entry books, radix-16 partial products shared by 320 lanes, and indices fixed at fabrication. We claim the combination and its verified implementation, not the pieces.

Speech recognition hardware. Moonshine [25] is an encoder–decoder ASR transformer whose encoder cost scales with audio length. It reports lower compute than Whisper [47] tiny at equal error rate, and later variants add streaming encoders [29].

Dedicated ASR chips have used WFST and DNN–HMM decoders [44, 45] or attention accelerators within an SoC [53], and identified memory bandwidth as the dominant power cost. Always-on keyword-spotting chips reach sub-microwatt power [49, 17], but only by restricting the task to a few words. RouteVoice targets open-vocabulary transcription and removes weight fetch altogether.

Transformer accelerators and energy references. Programmable transformer processors at 12–28 nm report efficiencies in the tens of TOPS/W [56, 62, 54]. Groq [1] and Cerebras [31] keep weights in on-chip SRAM at scale. Microcontroller NPUs [3, 51] are the commercial baseline for edge voice.

Energy per operation is strongly node- and voltage-dependent [24, 52]. Our 180 nm, 3.3 V figures should be compared with these only after normalisation (section 6).

Open silicon and AI-driven design. Our flow builds on the following open tools:

- OpenROAD [2] and OpenLane [48], now continued as LibreLane [30];
- Yosys [66], Magic [13], KLayout [28], Netgen [14] and Verilator [50];
- the open GF180MCU PDK [18].

Low-cost shuttles such as Tiny Tapeout [58] and wafer.space [60] make fabrication on open nodes affordable.

LLMs have been applied to RTL generation [33, 34, 9], domain assistance [32], and flow orchestration [22, 26, 63,

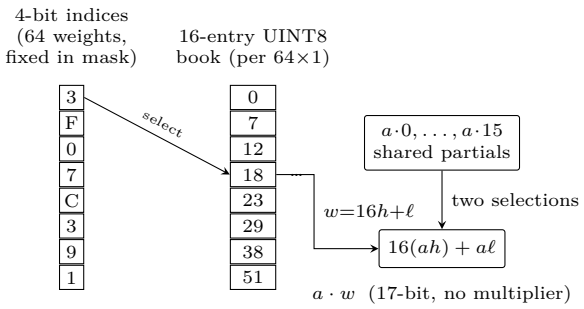


Figure 1: K64 weight representation. Each weight is a fixed 4-bit index into a 16-entry UINT8 book shared by 64 weights of one output column. The selected value's two nibbles select two entries from a 16-entry table of partial products of the current activation a. That table is computed once and shared by all 320 lanes.

37]. One accumulator CPU designed through conversation reached tape-out [8]. Recent studies evaluate coding agents on complete RTL-to-GDS flows [12] and use them to improve OpenROAD itself [16]. These studies use benchmark-scale designs. We contribute an experience report: a multi-agent team carrying a 10^7-parameter-class accelerator through physical design, with its failure modes and costs recorded.

3 Weight Representation

3.1 Model

We use Moonshine Tiny Streaming (English) [25], a 34 M-parameter encoder–decoder ASR transformer. It has width 320, six encoder and six decoder layers, eight heads of dimension 40, and a 32,768-token vocabulary with tied embedding and output head. Its audio frontend has no FFT or mel stage. Per 5 ms group of 80 samples it applies:

1. mean and variance normalisation;
2. an asinh compression;
3. a learned 80→320 projection;
4. two stride-2 convolutions, giving 50 Hz features.

The released model is already integer-quantized (UINT8 weights with per-column zero points and FP32 scales). Our representation starts from those integer weights.

3.2 K64: four-bit indices into local codebooks

Each weight matrix is partitioned into groups of 64 consecutive input rows for one output column. A group stores:

- a book of 16 UINT8 representative values;
- one 4-bit index per weight, selecting a representative.

We call this format K64 (fig. 1). Across the model it yields 98 packed matrices, 31,457,280 four-bit selections (15.7 MB of indices) and 491,520 books.

Table 1: Word error rate (WER) of K64 versus the original Moonshine Tiny on the single reserved test.

Condition	K64 WER	Δ vs. original	95% CI of Δ
clean	4.49%	+0.23	[−0.83, +1.48]
other	13.63%	+0.89	[−1.51, +3.19]
noise (10 dB)	15.77%	+1.84	[−0.77, +4.48]

Because the books hold UINT8 values, the representation slots into the existing integer pipeline unchanged. Zero-point correction, FP32 scaling and bias are applied exactly as in the source graph. The integer weights the chip computes with are *defined* by the books, and our hardware is bit-exact with respect to this frozen K64 model, not the original weights. In the selected 1280×320 projection, 291,832 of the 409,600 weights differ from the original.

Accuracy. Accuracy was settled before any hardware work, under a pre-registered protocol. Candidates were tuned on calibration data, checked once on a development split, and then evaluated once on a reserved LibriSpeech-derived test set [41]: 80 utterances from 40 held-out speakers, plus a 10 dB white-noise copy of the clean set. The gates were an absolute WER ceiling and at most +2 points over the original model in each condition.

K64 passes all three gates (table 1). The paired speaker-cluster bootstrap intervals for "other" and "noise" extend beyond +2 points, so we read this as a point-estimate pass on a small test, not a population guarantee. Coarser alternatives failed the same gates: per-tensor 4-bit, 32-column grouped, and K128 (books shared by 128 rows). The per-tensor and grouped candidates lost up to +9.4 points on noisy speech.

3.3 Shared partial products

The engine never multiplies a weight. Write each 8-bit representative as $w = 16h + \ell$ with $h, \ell \in [0, 15]$. Then

$$a \cdot w = 16\,(a \cdot h) + a \cdot \ell. \tag{1}$$

For each input activation a (signed 9-bit), the engine forms the 16 products $a\cdot0, \ldots, a\cdot15$ once, as 13-bit values. Every one of the 320 output lanes then needs only two selections from this table, a shift and an add to obtain $a \cdot w$; the result fits in 17 bits.

The fixed index picks the book entry, the book entry's nibbles pick two partial products, and the multiply becomes wiring plus a small adder. This mirrors the shared-product idea described in hardwired-weight patents [4, 7]. We implement it independently from the public documents, and make no claim about any production implementation.

3.4 What is fixed, and what is not

In a fabricated RouteVoice, every 4-bit index is fixed by the mask. Our physical bank uses four mask-programmed access transistors per selection, with the bit pattern set by Via1 positions, followed by a precharged read path per bit. We read the bit pattern back from the GDS and checked it against the packed model.

The books, zero points, scales and biases are fixed constants in the current RTL. Activations, attention state, the audio window and decoder caches are mutable and need SRAM. The measured index bank costs 5.54 µm^2 per selection, including readout and local wiring, against about 51 µm^2 per bit for the open GF180 SRAM macro. Storing every weight in open SRAM would take roughly 6,400 mm^2; the 81 index banks need 174.7 mm^2.

4 Architecture

4.1 The projection engine

The unit of physical work is one 1280 × 320 projection: the decoder feed-forward output matrix, the largest shape in the model. The engine (fig. 2) is a five-state machine:

LOAD accepts one signed 9-bit activation, adds it to a running activation sum, and registers its 16 partial products.

INDEX reads one 1,280-bit row of the index array (320 four-bit selections) and maps each index through its book to a UINT8 representative.

ACCUM adds, in each of 320 lanes, the two selected partial products (one shifted) into a signed 27-bit accumulator.

PREP (added in section 7) fills per-group output registers before output begins.

OUTPUT emits one column per cycle. Each value is the accumulator minus activation sum × zero point, passed through a three-stage FP32 multiply–add (Berkeley HardFloat [21]) that applies the activation scale, the weight scale and the bias with independent round-to-nearest-even steps.

One projection takes 3 × 1280 cycles of LOAD/INDEX/ACCUM and 320 output cycles, plus pipeline fill: 4,165 cycles in the final design (4,163 originally). That is 409,600 multiply-accumulates, an average of 98 MAC per cycle.

Two structural changes made the engine physically implementable:

1. **Fanout.** Row-index and partial-product nets originally drove up to 11,359 loads. We duplicated the row registers ten times and capped synthesis fanout at 16.
2. **Local partials.** Each 32-lane group gets its own copy of the partial-product registers, so the 208-bit partial-product bus no longer crosses the die (section 7).

Both changes are cycle-identical on the full test bench.

4.2 From one engine to a recognizer

The full recognizer RTL connects:

- the PCM input and frontend;
- the six-layer encoder;
- an adapter that produces cross-attention keys and values;
- a greedy six-layer decoder that ends at the end-of-sequence token.

Everything runs on one common clock. Each weight service answers lookups through index and book ROMs. The top level has no weight inputs: its 31 runtime inputs are audio, clocks, resets, handshakes and configuration.

Throughput. A dense one-pass estimate of the workload is about 0.6 GMAC/s:

- the encoder: about 7.4 M MAC per frame at 50 frames/s;
- the decoder: about 21.5 M MAC per token at about 4 tokens/s;
- the keys and values produced for cross-attention.

One engine at 150 ns sustains 0.66 GMAC/s. However, the native streaming controller re-processes encoder frames and regenerates cross-attention keys and values. Measured on real traces, that controller needs 10–90 engine-equivalents at a 120 ns clock. Real-time operation therefore depends as much on scheduling as on raw MAC rate, and we treat it as open (section 10).

4.3 Chip plan

All 98 matrices map to 81 pitch-matched index banks (174.7 mm^2) plus four bank-driver control strips (56.6 mm^2). Abstract floorplans with power grids give two candidate chips:

- **One die** of 25 × 31 mm, which needs a denser SRAM than the open GF180 macro (we assumed 12 µm^2/bit);
- **Two dies** of 25 × 29 mm, built only from existing open parts, with no change to the numerics.

The two-die variant routes and passes IR drop at the measured engine power, but fails an earlier, more pessimistic assumed-load IR screen.

About 1.09 MB of mutable state (activation banks, key/value caches, history and scratch) dominates the remaining area. Synthesized as flip-flops, it alone puts the logic at 1,198 mm^2 (section 6). Mapping it onto SRAM macros is the largest open area item.

5 Methodology

5.1 Proof levels and evidence rules

Every claim in the project names an artifact and a proof level. The levels run from P0 (idea) through software, numerical reference, RTL block, connected RTL, openflow physical design, FPGA, foundry signoff and silicon, up to P9 (product). A claim at one level never stands in

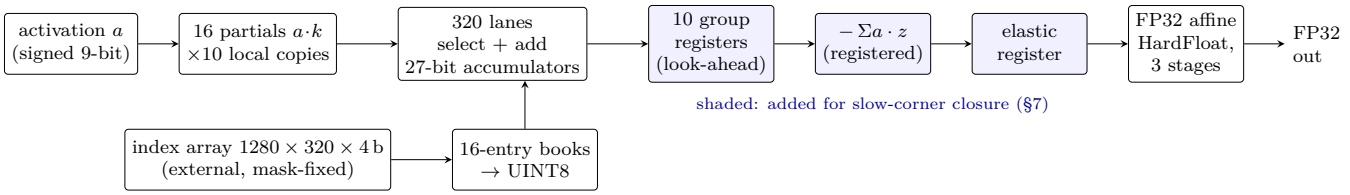


Figure 2: The 1280 × 320 projection engine. The index array sits outside the engine's physical top. One projection takes 4,165 cycles: 1,280 rows of LOAD/INDEX/ACCUM, one PREP cycle, and 320 output cycles.

for another. A routed engine is not evidence of recognizer accuracy, and a correct index pattern is not evidence of a correct weight.

Three rules governed the work:

1. A result is recorded as PASS only after the lead re-ran it or read the actual tool output. Reports from worker agents are recorded as "claimed" until then.
2. Failures are kept, not overwritten. Each workstream keeps a `FAILURES.md` next to its `RESULTS.md`.
3. Inputs and outputs are pinned by SHA-256: model tensors, packed indices, books, tool binaries and run databases. Off-machine backups are re-hashed after upload.

5.2 Step-by-step procedure

1. **Freeze the model and the oracle.** Pin the Moonshine release by hash. Build an independent NumPy reference over the executable tensor bytes, and capture boundary tensors, logits and tokens for each test clip.
2. **Choose the representation in software.** Search quantization candidates against pre-registered WER gates, ending with one reserved test (section 3).
3. **Write a numerical contract for one projection.** Fix bit widths, cycle counts and an exact integer and FP32 oracle. Bound accumulators on the actual matrix.
4. **Build and verify the RTL engine.** The bench uses 54 vectors: one captured from speech and 53 integer fixtures, giving 17,280 integer and 17,280 FP32 outputs. It also exercises stalls and four mid-vector resets.
5. **Connect the whole recognizer.** Move every weight on chip, first as byte ROMs, then as index+book services. Require identical tokens, hashes and clock counts on every change.
6. **Lay out the weight banks.** Generate pitch-matched mask-programmed banks. Then run DRC, read the bit pattern back from GDS against the packed model, and run LVS against a source-derived netlist. Include planted-error negative controls.
7. **Implement the engine physically.** Synthesize, audit fanout, run short global-route diagnostics, then do the full route, signoff (DRC, LVS, antenna, XOR) and all-corner extracted timing.
8. **Measure power from real activity.** Run a gate-level simulation of one real projection, rescale its switching trace to the clock period, and annotate it on the routed database with extracted parasitics.
9. **Check whole-design synthesizability** for both silicon and FPGA.
10. **Floorplan the chip** with abstract macros, power grids and IR-drop screens, then integrate the real banks and logic macros.

5.3 Open toolchain

All tools are open source:

- **Synthesis:** Yosys 0.62 [66], including `slang` for SystemVerilog.
- **Place and route:** OpenROAD [2] under LibreLane 3.0.11 [30].
- **Timing:** OpenSTA 2.7.
- **Layout and DRC:** Magic 8.3 [13] and KLayout 0.30 [28], using the full GF180 main deck.
- **LVS:** Netgen 1.5 [14].
- **Circuit simulation:** ngspice 47.
- **Functional and gate-level simulation:** Verilator 5.05 [50].

The PDK is GF180MCU-D [18] with its 7-track standard cells, characterised at TT (25 °C, 3.3 V), SS (125 °C, 3.0 V) and FF (−40 °C, 3.6 V), each with minimum, nominal and maximum RC. The toolchain is pinned in a Nix environment.

Compute was an Apple M4 Pro laptop (14 cores, 48 GB) and two rented CPU servers. The servers were used for about 47 hours in total and cost $34.82. All closure and power work after 22 October ran on the laptop.

5.4 Agent orchestration

The design work was carried out by AI coding agents under human direction:

- a lead agent (Claude Code) planned the work, wrote briefs, verified results and integrated them;
- up to six parallel worker agents (OpenAI Codex CLI) each owned one workstream, with a written brief, a private directory, and explicit limits on compute, memory and cloud spend;
- short-lived sub-agents handled read-only research and inventories.

Workstreams in the final push were:

- W10: engine routing and signoff;
- W11: index+book recognizer;
- W12: full synthesis;

- W13: weight banks;
- W14: logic macros;
- W15: two-die integration.

The human set goals and budgets and approved spending. The lead enforced the evidence rules. Several practices proved necessary:

- **Verify process state, not worker reports.** Workers sometimes reported jobs as "continuing" after their processes had exited. One such case left a rented server idle for about 21 billed hours.
- **Publish a shared resource schedule.** A container's real memory limit (64 GB) was far below what the operating system reported (755 GB). After one out-of-memory kill, the lead published a priority order and a memory ceiling for all workers.
- **Keep a verified external backup before tearing down cloud machines.** A 53 GB checksum-verified backup let the rented server be deleted while results were still being produced; routed power and every later closure run started from the restored data.
- **Have the lead do the critical-path engineering.** Timing closure (section 7) ran as short, verified iterations by the lead after the workers had exited. This was faster than delegating.

Section 8 quantifies where time and money went.

6 Results

We report three kinds of evidence, in decreasing order of completeness:

- functional bit-exactness of the whole recognizer;
- a fully routed and signoff-checked projection engine;
- synthesizability of the whole recognizer for both silicon and FPGA.

Unless noted, every number was re-read from tool output by the lead before it was recorded (section 5). Source files are listed in appendix A.

6.1 Bit-exact recognition from index and codebook weights

The full recognizer RTL decodes the clip "Switch on the lights" to the token sequence $[28176, 373, 278, 26068, 2]$. Every weight is served as a 4-bit index into a 16-entry codebook, one codebook per 64-weight group. All 18 checked output hashes match the reference, and the run takes 312,578,649 clocks, identical to the earlier byte-ROM design. Two further clips ("Plant present." and "Seasonable.") also match on all 18 hashes.

Table 2 shows the storage this needs. Weight storage falls from 28.64 MB to 17.50 MB (−38.9%), and decoding the stored form back reproduces all 28,641,280 original bytes with zero mismatches. Exhaustive RTL checks of the lookup responders cover 565,960 encoder/decoder packets, 2,074,560 frontend packets and 163,840 position packets.

Table 2: On-chip storage of the full recognizer after index+codebook conversion.

Component	Bytes
4-bit indices (78,970,880 bits)	9,871,360
16-entry UINT8 books (one per 64 weights)	4,935,680
Affine INT8 frontend (lossless)	2,073,600
Metadata (scales, zero points, biases)	616,960
Total (original: 28,641,280)	17,497,600

The test set is small. It establishes that the hardware representation is *exact* with respect to the quantized model, not that the quantized model is accurate. Accuracy of the codebook quantization itself was measured in software on held-out speech (section 3).

6.2 Routed projection engine

The 1280×320 engine top contains the arithmetic and output stages for 409,600 four-bit selections. The fixed index array sits outside it and is supplied by the test bench. Physical design used LibreLane on GF180MCU-D (7-track cells, 5 metal layers) at 150 ns.

The final design is "fix 2" in section 7, after an antenna close-out pass. On a 22.84 mm^2 die it has 1,146,232 standard cells (54.0% utilization) and 68.9 m of routed wire. Signoff results:

- detailed routing: zero violations;
- Magic DRC: zero violations;
- KLayout DRC (main GF180 deck): zero violations;
- GDS XOR between the Magic and KLayout streams: zero differences;
- antenna: zero violating nets or pins;
- LVS (Netgen): circuits match uniquely, with zero device, net, pin or property differences.

Setup and hold are met in all nine PVT/RC corners; the slowest is SS with worst-case RC at +8.10 ns, and the worst hold slack is +0.32 ns. Slew and capacitance-limit warnings remain in every iteration, at about 390,000 and 21,000 at TT; they do not cause setup or hold failures and are listed as open work.

Each projection takes 4,165 cycles, so one engine at 6.67 MHz sustains about 0.66 GMAC/s. That meets the recognizer's real-time load of about 0.6 GMAC/s.

Power. Activity comes from a zero-delay gate-level simulation of one real-speech projection on the synthesized netlist. All 320 FP32 outputs of that run are bit-exact. Annotated against the final routed database and extracted parasitics, the engine draws 0.188 W at TT and 150 ns: 142 mW combinational, 24 mW sequential and 21.5 mW in the clock tree. That is 117 µJ per projection, or 287 pJ per MAC.

For orientation only, we scaled Horowitz's 45 nm, 0.9 V operation energies [24] by CV^2 to 180 nm and 3.3 V, a factor of about 54. That gives roughly 11 pJ for an 8-bit

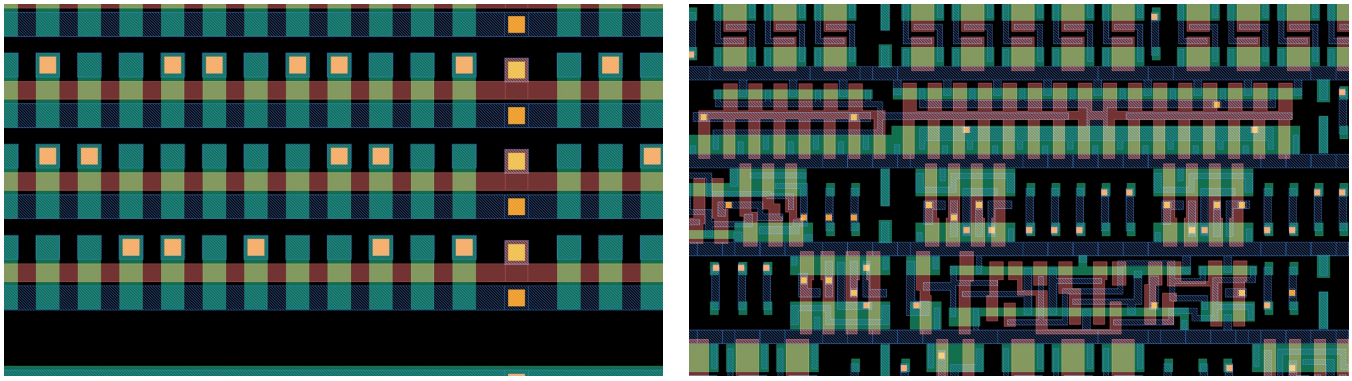


Figure 3: Layout close-ups rendered from the GDS (Via1 amber, Metal1 blue, poly and diffusion beneath; fill hidden). *Left:* 10.4 µm-wide window of weight bank `W13_ENC0_B00`. Each 4-bit index uses four mask-programmed access transistors; the irregular Via1 pattern over the regular cell grid sets the stored bits. *Right:* 29.5 µm-wide window of the routed 1280 × 320 engine (fix 2): standard cells of the select-and-accumulate datapath. Zoom sequences from full die to cells are in the release package (appendix A).

multiply and 250 pJ for an FP32 multiply–add. This is our own first-order estimate, not a measurement. Our per-MAC figure amortises one FP32 affine operation over 1,280 integer MACs, so it is dominated by the 320-lane integer select-and-accumulate datapath and its clock tree, not by floating point. A per-module breakdown is not available from the flattened routed netlist; we leave it to a hierarchical power run.

The pre-route estimate for the same engine was 162 pJ/MAC; that figure had no clock tree and only placement parasitics. Pin annotation was 67.5%; the remaining pins are cells inserted after synthesis, which the analyser assigns default activity. The trend across iterations (241→256→287 pJ/MAC) therefore mixes real design change with falling annotation coverage.

6.3 Whole-recognizer synthesizability

Synthesizing the complete recognizer flat failed for both silicon and FPGA. Synthesis tools stalled for hours in memory and flip-flop optimisation passes, and the FPGA run crashed. We therefore black-boxed the weight ROMs and synthesized the remaining 30,545 elaborated modules as 4,457 distinct units, each under a 6 GB memory limit. All 4,457 units pass for a generic library, for GF180, and for the Lattice ECP5 FPGA family.

The results are informative mainly as an upper bound. The GF180 netlist is 1,198 mm^2 (48.4 M cells, 8.07 M flip-flops), because all 8,169,778 bits of mutable state (buffers, caches, normalisation state) become flip-flops when no SRAM macros are inferred. That is 19× the 63 mm^2 logic budget of our floorplan. The ECP5 mapping needs 30.4 M LUT4s, 1.34 M flip-flops, 1,247 DSP blocks and 118 BRAMs, far beyond any device in the family. The weights alone (139,980,800 bits in K64 form) also exceed on-FPGA memory by more than an order of magnitude.

6.4 Toward a full chip

Several physical parts of the full chip exist but are not integrated:

- **Weight arrays.** All 98 weight matrices (31,457,280 selections) are laid out as 81 pitch-matched index banks totalling 174.7 mm^2. Every bank is DRC-clean, and full LVS passes on six representative banks. A single bank measures 5.54 µm^2 per selection including readout.
- **Bank drivers.** The four bank-driver control strips are DRC- and LVS-clean and meet 150 ns in all nine corners.
- **Logic macros.** Six small logic macros pass block-level signoff.
- **Two-die top level.** Two 25 × 29 mm dies are routed with 48 real banks and 4 logic macros. Top-level wiring is DRC-clean, but the dies fail an assumed-load IR-drop check, and full-chip DRC, LVS and timing have not been run.

Section 10 discusses what remains.

7 Case Study: Closing the Slow Corner

Timing closure of the 1280 × 320 engine illustrates both how the open flow behaves at scale and why architectural fixes beat tool tuning. Table 3 summarises each iteration. Every row is an extracted-parasitic (RCX/SPEF) static timing result at a 150 ns clock (6.67 MHz).

Congestion first. The first full placements could not be routed. Global routing reported 3.37 M overflowed resources, because one set of radix-16 partial products fed all 320 lanes across the die. Giving each 32-lane group its own cycle-identical copy of the partial-product registers ("local partials") cut overflow to 260 k, and then to

Table 3: Engine closure iterations. Setup slack (worst endpoint) is in ns at a 150 ns clock with extracted parasitics. Energy uses activity-annotated routed power at TT. Every routed run is clean on routing DRC, Magic DRC, KLayout DRC and LVS; antenna counts are open nets after repair.

Run	GRT ovf.	TT	SS nom	SS max	Antenna	pJ/MAC	What changed
First route (var. C)	575	−72.9	−251	−307	130	241	local partials; no post-placement repair
Closure ECO 1	–	+55.0	−40.0	−55.5	4	256	post-route repair; antenna diodes
Closure ECO 2	–	+56.7	−36.2	−53.1	4	–	more aggressive repair
Fix 1 (stopped)	3,206	–	–	–	–	–	register before FP32 stage
Fix 2	9	+67.6	+21.6	+8.3	11	287	+ split 320:1 select
Fix 2, final	**9**	**+67.5**	**+21.4**	**+8.1**	**0**	**287**	+ 10 antenna-repair rounds

575 once post-placement repair was disabled. Detailed routing then converged to zero violations.

Tool-level closure plateaus. With routing clean, the typical corner (TT, 25 °C, 3.3 V) met timing with +55 ns of slack. The slow corner (SS, 125 °C, 3.0 V) failed by −40 ns (nominal RC) and −55.5 ns (worst-case RC). We ran two engineering-change (ECO) passes, re-entering the flow after global routing. They used more aggressive repair: a 45 ns setup-slack margin, buffering allowed up to 50% of cells, 100% of failing endpoints targeted, and 20 antenna-repair iterations. These passes recovered only about 4 ns (to −36.2/−53.1 ns), and the *same* 30 endpoints failed in both. We stopped tuning there: the failing paths were too deep for one cycle at SS, whatever the repair settings.

Diagnosis. All 30 failing paths start at the column register. They then pass through:

- the 320:1 accumulator select;
- the zero-point lookup and its correction multiply and subtract;
- the integer-to-floating-point conversion.

They end in the first register stage of the FP32 affine block.

The two first-order models disagreed on the size of the problem. Using estimated parasitics after global routing, the pre-fix design showed +36 ns of SS slack. Its routed result was −36 ns, a 72 ns gap. The gap comes from the die-wide 320:1 select: its fan-in wires are long and only appear once detailed routing is done.

Fix 1: one elastic register. We first inserted an elastic valid/ready register between the integer engine output and the FP32 affine stage; weight scale, bias and column index move through it with the data. Throughput is unchanged, and latency grows by one cycle (4,163 to 4,164 cycles per projection). The 54-vector bench still matched all 17,280 FP32 outputs bit-exactly. Estimated SS slack on the column path rose from +36 to +61 ns. Applying the 72 ns gap leaves an expected routed slack of about −10 ns, still failing. Global-route overflow also rose to 3,206. We therefore stopped this run at detailed routing.

Fix 2: split the select with look-ahead group registers. Each 32-lane group now registers its own lane for the *look-ahead* column, so the output path sees only a 10:1 select of group registers. The look-ahead column is $c{+}1$ on an output handshake and c otherwise. The zero-point correction is registered the same way. A single `PREP` state between the last accumulation and the first output fills these registers. The change is generated by a script from the original RTL. It adds one more cycle (4,165 per projection), keeps one output per cycle, and again passes all 54 vectors bit-exactly.

The physical effect was large. Global-route overflow fell to 9, and detailed routing converged in about 75 minutes. Signoff is clean: zero routing, Magic and KLayout DRC violations, a unique Netgen LVS match, and zero GDS XOR differences; 11 antenna nets remain open, against 4 before. All nine corners (TT/SS/FF × min/nom/max RC) meet setup and hold at 150 ns. The worst case is +8.34 ns at SS with worst-case RC, against −53.1 ns before.

Antenna close-out. The 11 open antenna nets were not a design problem. The router's post-route antenna repair had stopped at its default three rounds while still converging (199, 41, 24, then 11 violating nets). Rerunning from the same detailed-routing input with up to ten rounds, and nothing else changed, took the count to zero: 8, 5, 5, 5, 1, 1, then 0, adding about 1,000 diodes (7,452 in total). The full signoff was then repeated: routing, Magic and KLayout DRC, XOR, disconnected-pin and power-grid checks are all zero, and LVS again matches uniquely. Timing moved by less than 0.3 ns in every corner (worst +8.10 ns setup, +0.32 ns hold), and routed power with the same activity trace is unchanged. The run took 6.7 hours on a laptop, two of them in KLayout DRC.

Cost of closure. Routed, activity-annotated power for fix 2 is 0.188 W at 150 ns at TT. That is 117 µJ per projection, or 287 pJ per MAC, about 12% more than the ECO-1 design (256 pJ/MAC). The increase comes from the ten 27-bit group registers, the registered correction, and extra repair buffering. Both power runs annotate about 68% of pins from a zero-delay gate-level trace of

one real-speech projection. The unannotated pins belong to repair, antenna and diode cells that do not exist in the synthesized netlist.

Lesson. Two passes of repair-setting tuning, about 17 hours of compute, did not move the failing endpoints. One RTL change of under 40 lines, verified bit-exactly on the full bench within minutes, closed every corner and also removed the routing congestion. The general rule we draw is to pipeline wide selects and long fan-in paths at RTL before physical design. A small set of path-level timing estimates, calibrated against one fully routed run, is enough to decide when to stop tuning.

8 Making It Faster and More Efficient

We separate improvements to the *design*, which change area or energy, from improvements to the *flow*, which change how long it takes to reach a verified design.

8.1 Design

Map mutable state to SRAM. The full-recognizer synthesis puts 8.17 M mutable bits into flip-flops. At 1,198 mm^2, that is the single largest gap between our RTL and a manufacturable chip. Open GF180 SRAM macros and a bridge design that replays the full clip with 160 macros already exist. Integrating them is engineering work, not research.

Time-share datapaths. The RTL instantiates separate arithmetic for each service. A small number of projection engines, scheduled over all matrices, would replace most of that logic. The scheduling question is the one raised in section 4: how many engine-equivalents real-time streaming really needs.

Lower the activity, not just the clock. Combinational logic draws 75% of engine power (142 mW of 188 mW). The 320 accumulators and partial-product selectors toggle on every accumulation cycle. Three changes would each cut that activity:

- gate the clock of lanes whose selected product is zero;
- narrow the accumulators to the bound proven on the actual matrix;
- operand-isolate the FP32 stage outside the output phase.

We have not yet measured these savings; the real-speech activity trace makes them straightforward to evaluate.

Voltage and node. GF180's 3.3 V 7-track library is the least energy-efficient option we could have chosen. The engine meets 150 ns with +67 ns of slack at TT. A lower supply (where the library supports it), or the 9-track 5 V cells avoided, would trade that slack for energy. First-order CV^2 scaling [24] suggests that moving to 28 nm at about 0.9 V would cut energy per operation by one to two orders of magnitude. Real gains are smaller, because wire capacitance scales worse than transistors. Index density would rise by a similar factor. The whole model's index banks (175 mm^2 here) would then fit in a few tens of mm^2.

8.2 Flow

The project's own history shows where the time went and what would have saved it.

1. **Route a short diagnostic first.** Three-iteration global-route diagnostics took 25–61 minutes and found the congestion cause. Full routing attempts without that diagnostic burned about 11 hours in global routing and about 1 hour per detailed-routing iteration without converging. Only start a full route once the diagnostic overflow is small: overflow of 575 routed in hours, while 260 k never did.
2. **Fix wiring and depth in RTL, not with die area or repair settings.** A die 1.6 times larger cut overflow by only 24%, while local register copies cut it by 92% at the same area. Two aggressive repair passes moved slow-corner slack by 4 ns, while one RTL pipeline change moved it by 58 ns and reduced overflow from 575 to 9. When the same endpoints fail after repair, pipeline.
3. **Pick a reachable clock early.** Repairing toward 20 ns grew area by 26% and never finished, because the datapath was about 145 ns deep. At 150 ns the design closed.
4. **Audit fanout at synthesis.** Capping fanout at 16 cut estimated power 10.8× and turned placement from hours into minutes.
5. **Calibrate pre-route timing once.** Estimated slow-corner slack after global routing was 72 ns optimistic for die-spanning paths. One calibration against a routed result was enough to stop a doomed run early (fix 1) and to predict that fix 2 would close.
6. **Check that repair loops converged, not just stopped.** The last 11 antenna violations remained only because the router's post-route repair loop ends after three rounds by default, while it was still halving the count each round. Raising the limit to ten cleared them with no design change. Read iteration logs, not just final counts.
7. **Signoff dominates wall time.** KLayout DRC takes about 2 hours on every run, comparable to detailed routing. Run signoff only on candidates that already meet timing, and parallelise or tile the DRC deck.
8. **Most of the work fits on a laptop.** Every engine flow (4–9.5 hours) ran on the laptop at zero cloud cost. Whole-design synthesis needed big memory only when attempted flat; per-module synthesis under 6 GB per

process completed every unit.

9. **Bill only for active compute.** Roughly 21 of the 47 rented hours were idle. An idle-CPU auto-stop, plus checking worker process state rather than worker reports, would have saved over a third of the cloud spend (about $13 of $34.82).
10. **Check format compatibility early.** An activity-file syntax mismatch between simulator and timing analyser, and a cell-name format bug in a buffering script, each cost a full run cycle.

In aggregate, the engine went from its first RTL contract (13–14 October) to a routed, all-corner-closed layout clean on DRC, LVS and antenna checks (24 October) in eleven calendar days, including a five-day pause. The final timing fix, including the abandoned first attempt, took about seven hours from diagnosis to all-corner routed timing. With the practices above applied from the start, we estimate the same result could be reached in two to three days.

9 Future Scope: Local LLMs and Multi-Purpose Transformer Silicon

RouteVoice is one model on one engine. This section sets out what the same techniques imply for local language models and for silicon that serves more than one transformer. Unlike the rest of the paper, these are projections, not measurements.

9.1 From recognizer to voice assistant

A useful local voice device chains speech-to-text, a small language model, and text-to-speech. All three are transformer stacks built from the same few operators: dense projections, attention, normalisation and elementwise activations.

The projection engine here is model-agnostic below the index fabric. It consumes a stream of signed activations and a fixed table of 4-bit selections, and emits FP32 results. A second model needs a second index fabric, not a second engine design.

The open question is *state*, not weights. A recognizer's mutable state is its audio window and decoder cache. An autoregressive LLM adds a key/value cache that grows with context, which belongs in SRAM or external DRAM, never in metal. For scale, a 0.5 B-parameter LM such as Qwen2.5-0.5B [46] (24 layers, two key/value heads of dimension 64) keeps 6,144 key/value values per token. At 8 bits, a 2,048-token context needs about 12.6 MB, roughly twelve times all of RouteVoice's mutable state (8.17 Mbit).

9.2 Hardwired weights at larger scale

Fixed-index storage has a clear scaling trade:

- **Benefits:** no weight fetch energy, no DRAM bandwidth wall, and silicon that cannot be repurposed by tampering with weights.
- **Cost:** a mask set per model.

Commercial hardwired-LLM efforts [5] accept that cost at the frontier for throughput.

Our results suggest a complementary point at the low end, and they let us size it.

Throughput. A dense decoder-only LM of N parameters needs about N multiply-accumulates per generated token. One of our engines sustains 0.66 GMAC/s at 6.67 MHz. A 0.5 B-parameter model would therefore generate about 1.3 tokens/s per engine, and ten engines about 13 tokens/s, which is enough for a spoken reply.

Area. The binding constraint is index area, not compute. At our measured 5.54 µm^2 per selection, 0.5 B selections need about 2,800 mm^2 at 180 nm, several reticles.

Node scaling. Ideal area scaling to 28 nm is $(180/28)^2 \approx 41\times$, which would bring the same banks to roughly 70 mm^2. Peripheral circuits scale worse than this, so the figure is an optimistic bound. Figure 4 plots the measured points and a conservative 20–40× band.

Where this leaves the low end. A speech model of tens of millions of parameters (this work) fits a mature node today. A sub-billion-parameter assistant LM needs a mid-range node to become a single affordable die.

9.3 Making hardwired silicon multi-purpose

Hardwiring trades flexibility for efficiency. Several mechanisms can recover flexibility without returning to fully programmable weights:

1. **Mutable codebooks over fixed indices.** The 4-bit indices are fixed; the 16-entry books (4.9 MB for the services we converted, about half the index bytes) could instead live in SRAM. Then one die could be re-tuned after fabrication: recalibrated, rescaled, or fine-tuned within the span its fixed indices allow. At open-SRAM density that costs far more area than the books do as ROM, so it suits a model family whose indices are shared and whose books differ. Whether book-only fine-tuning recovers useful accuracy is an experiment we have not run.
2. **Low-rank adapters in SRAM.** A fixed base model plus small SRAM-resident adapters (LoRA-style) on selected projections supports task or speaker adaptation. Our 160-SRAM bridge and producer adapter work is a first step toward this partition.
3. **Shared backbones.** Distilling several tasks onto one shared encoder lets one fixed fabric serve many heads: transcription, keyword spotting, intent, language identification. Only the small task heads stay mutable.

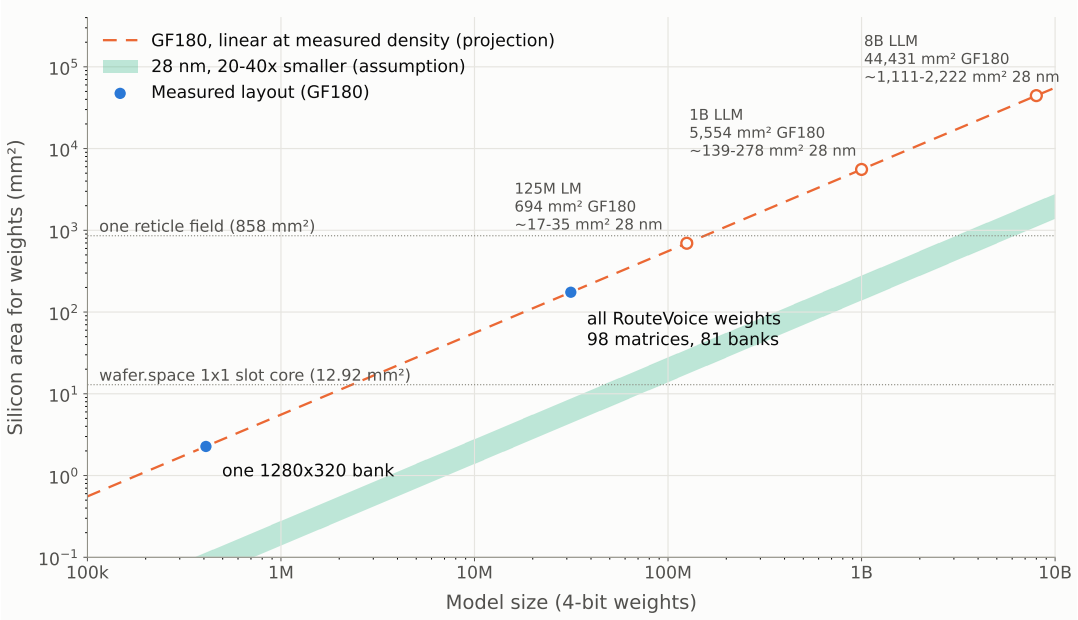


Figure 4: Weight-array area against model size. Filled points are measured GF180 layout: one 1280×320 bank (2.27 mm^2) and all 98 RouteVoice matrices (31,457,280 four-bit weights, 174.7 mm^2, i.e. 5.55 µm^2 per weight including readout). The dashed line extends that density linearly; the shaded band assumes a 20–40× area reduction at 28 nm. Both are projections, not measurements. Weights only: compute engines, activation and KV memory, and I/O are excluded.

4. **Family dies.** A mask set can hold several small models whose index fabrics share one engine and I/O ring, selected at run time. Area grows linearly in model bytes, but NRE is paid once.

9.4 Economics

The economic case rests on volume. At mature nodes, mask sets cost far less than at leading edge, and multi-project shuttles cost less still [60]. AI-assisted design also shortens the model-to-GDS loop, which this work demonstrates in miniature. For scale, a wafer.space GF180 shuttle slot of about 20 mm^2 costs $8,000 for 1,000 dies [60]. That fits one engine or test structure, not our two 725 mm^2 dies. A full recognizer at this node therefore needs a dedicated mask set, or a newer node where it shrinks to tens of mm^2. We do not have a verified full-mask quotation and give none. Against this stand the costs of a fixed model: it cannot follow a fast-moving model frontier, and long-context state must still live in memory. The natural first markets are therefore products whose model can stay fixed for the life of the device: appliance voice control, wearables, toys, industrial sensors.

10 Limitations

We state the boundaries of the evidence explicitly.

- **No silicon.** Every physical result comes from open-source tools on an open PDK. None has passed foundry signoff or been fabricated.
- **Engine, not chip.** Routing, signoff, timing and power cover one 1280×320 projection engine, without its index array. The two-die chip is floorplanned and partly integrated. It has no full-chip DRC, LVS or timing, and it fails an assumed-load IR-drop check.
- **Open signoff items.** The final engine is clean on DRC, LVS and antenna checks with the open GF180 decks. Slew and capacitance-limit warnings remain in every iteration; they cause no setup or hold failure, but a foundry signoff would require them fixed.
- **Power fidelity.** Activity comes from one real projection, simulated without delays (so glitches are not modelled), with 67.5% of pins annotated. The rest are cells added during physical design and receive default activity. Power for the full recognizer has not been measured.
- **Timing simulation.** Post-layout simulation replayed all vectors on the routed netlist without delays. None of the simulators available to us supports SDF timing checks at this scale.
- **Real-time operation is unproven.** One engine meets a dense one-pass MAC estimate. The native streaming controller's measured workload needs 10–90 engine-equivalents, and the frontend's per-chunk deadline implies a far higher clock than the engine's. A service schedule that meets real time has not been built.
- **Accuracy.** The bit-exact results are against the frozen K64 model. K64's accuracy rests on one reserved test of 80 utterances from 40 speakers, whose

confidence intervals cross the +2-point gate for noisy and "other" speech. Streaming quality on live audio was not measured.

- **Scale of RTL.** As written, the recognizer is about 1.2 cm^2 of GF180 logic, because its mutable state is synthesized as flip-flops.
- **Relation to Taalas.** We implement ideas described in public patent documents. We make no claim about any company's production design.

11 Conclusion

RouteVoice shows that a real speech transformer can be expressed entirely as fixed 4-bit indices and small codebooks, without leaving the accuracy envelope of its source model. It also shows that the resulting arithmetic engine can be carried through an all-open-source flow to a routed, DRC-clean layout that closes timing in every corner of a 180 nm process.

The most transferable lessons are methodological. The fixes that mattered were all at RTL: local register copies against congestion, and pipelining a die-wide select against slow-corner timing. They were found fastest by short, calibrated experiments, not by long tool sweeps. AI coding agents made it possible to run many workstreams in parallel at very low cost. They were reliable only when every claim was checked against the tools' own outputs.

The open work is concrete: mapping mutable state to SRAM, scheduling for real time, full-chip signoff, and silicon. Fixed-model silicon at mature nodes, built quickly with open tools and agents, is a credible path to private, low-power voice and language interfaces for everyday devices.

References

[1] Dennis Abts, Jonathan Ross, Jonathan Sparling, Mark Wong-VanHaren, Max Baker, Tom Hawkins, et al. Think fast: A tensor streaming processor (TSP) for accelerating deep learning workloads. In *ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pages 145–158, 2020. doi:10.1109/ISCA45697.2020.00023.

[2] Tutu Ajayi, Vidya A. Chhabria, Mateus Fogaça, Soheil Hashemi, Abdelrahman Hosny, Andrew B. Kahng, et al. Toward an open-source digital flow: First learnings from the OpenROAD project. In *Proceedings of the 56th Annual Design Automation Conference (DAC)*, pages 1–4, 2019. doi:10.1145/3316781.3326334.

[3] Arm Ltd. Arm Ethos-U55 NPU. Product page. URL: https://www.arm.com/products/silicon-ip-cpu/ethos/ethos-u55.

[4] Ljubisa Bajic. Hardware implemented codebook pointers. US Patent Application Publication US20260023566A1, January 2026. Assignee Taalas Inc. Filed 2025-07-18, priority 2024-07-19, published 2026-01-22. URL: https://patents.google.com/patent/US20260023566A1/en.

[5] Ljubisa Bajic. The path to ubiquitous AI. Taalas Inc. blog (Mission Log), February 2026. URL: https://taalas.com/the-path-to-ubiquitous-ai/.

[6] Ljubisa Bajic, Lejla Bajic, and Dragoljub Ignjatovic. Mask programmable ROM using shared connections. PCT International Publication WO2025217724A1, October 2025. Applicant Taalas Inc. Filed 2025-04-14, priority 2024-04-15, published 2025-10-23. URL: https://patents.google.com/patent/WO2025217724A1/en.

[7] Ljubisa Bajic, Dragoljub Ignjatovic, and Lejla Bajic. Large parameter set computation accelerator using configurable connectivity mesh. US Patent Application Publication US20250123802A1, April 2025. Assignee Taalas Inc. Filed 2024-09-13, priority 2023-10-11, published 2025-04-17. Pending. URL: https://patents.google.com/patent/US20250123802A1/en.

[8] Jason Blocklove, Siddharth Garg, Ramesh Karri, and Hammond Pearce. Chip-chat: Challenges and opportunities in conversational hardware design. In *ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*, 2023. URL: https://arxiv.org/abs/2305.13243, doi:10.1109/MLCAD58807.2023.10299874.

[9] Kaiyan Chang, Ying Wang, Haimeng Ren, Mengdi Wang, Shengwen Liang, Yinhe Han, Huawei Li, and Xiaowei Li. ChipGPT: How far are we from natural language hardware design, 2023. URL: https://arxiv.org/abs/2305.14019, arXiv:2305.14019.

[10] Yiming Chen, Guodong Yin, Mingyen Lee, Wenjun Tang, Zekun Yang, Yongpan Liu, et al. HiddenROM: A compute-in-ROM architecture to deploy large-scale neural networks on chip with flexible and scalable post-fabrication task transfer capability. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2022. doi:10.1145/3508352.3549335.

[11] Yiming Chen, Guodong Yin, Zhanhong Tan, Mingyen Lee, Zekun Yang, Yongpan Liu, Huazhong Yang, Kaisheng Ma, et al. YOLoC: Deploy large-scale neural network by ROM-based computing-in-memory using residual branch on a chip. In *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC)*, 2022. URL: https://arxiv.org/abs/2206.00379, doi:10.1145/3489517.3530576.

[12] Jinyuan Deng, Zhengrui Chen, Xufeng Wei, Tianyu Xing, Chenyi Wen, Qi Sun, and Cheng Zhuo. Can AI agents really complete RTL-to-GDS? lessons from benchmarking tool-interactive EDA workflows, 2026. URL: https://arxiv.org/abs/2607.17528, arXiv:2607.17528.

[13] R. Timothy Edwards. Magic VLSI layout tool. Open Circuit Design. Accessed 2026-10-24. URL: http://opencircuitdesign.com/magic/.

[14] R. Timothy Edwards. Netgen: netlist comparison (LVS). Open Circuit Design. Accessed 2026-10-24. URL: http://opencircuitdesign.com/netgen/.

[15] Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. Extreme compression of large language models via additive quantization, 2024. URL: https://arxiv.org/abs/2401.06118, arXiv:2401.06118.

[16] Amur Ghose, Junyeong Jang, Andrew B. Kahng, and Jakang Lee. Automated QoR improvement in OpenROAD with coding agents, 2026. URL: https://arxiv.org/abs/2601.06268, arXiv:2601.06268.

[17] J. S. P. Giraldo and Marian Verhelst. Laika: A 5uW programmable LSTM accelerator for always-on keyword spotting in 65nm CMOS. In *IEEE 44th European Solid State Circuits Conference (ESSCIRC)*, pages 166–169, 2018. doi:10.1109/ESSCIRC.2018.8494342.

[18] Google and GlobalFoundries. GF180MCU open source PDK. GitHub repository, 2022. URL: https://github.com/google/gf180mcu-pdk.

[19] Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A. Horowitz, and William J. Dally. EIE: Efficient inference engine on compressed deep neural network, 2016. URL: https://arxiv.org/abs/1602.01528, arXiv:1602.01528.

[20] Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding, 2015. URL: https://arxiv.org/abs/1510.00149, arXiv:1510.00149.

[21] John R. Hauser. Berkeley HardFloat. Release 1, July 2019. Accessed 2026-10-24. URL: http://www.jhauser.us/arithmetic/HardFloat.html.

[22] Zhuolun He, Haoyuan Wu, Xinyun Zhang, Xufeng Yao, Su Zheng, Haisheng Zheng, and Bei Yu. ChatEDA: A large language model powered autonomous agent for EDA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024. URL: https://arxiv.org/abs/2308.10204, doi:10.1109/TCAD.2024.3383347.

[23] Kartik Hegde, Jiyong Yu, Rohit Agrawal, Mengjia Yan, Michael Pellauer, and Christopher W. Fletcher. UCNN: Exploiting computational reuse in deep neural networks via weight repetition, 2018. URL: https://arxiv.org/abs/1804.06508, arXiv:1804.06508.

[24] Mark Horowitz. Computing's energy problem (and what we can do about it). In *IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, pages 10–14, 2014. doi:10.1109/ISSCC.2014.6757323.

[25] Nat Jeffries, Evan King, Manjunath Kudlur, Guy Nicholson, James Wang, and Pete Warden. Moonshine: Speech recognition for live transcription and voice commands, 2024. URL: https://arxiv.org/abs/2410.15608, arXiv:2410.15608.

[26] Zesong Jiang, Qing Zhang, Cheng Liu, Long Cheng, Huawei Li, and Xiaowei Li. IICPilot: An intelligent integrated circuit backend design framework using open EDA, 2024. URL: https://arxiv.org/abs/2407.12576, arXiv:2407.12576.

[27] Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W. Mahoney, and Kurt Keutzer. SqueezeLLM: Dense-and-sparse quantization, 2023. URL: https://arxiv.org/abs/2306.07629, arXiv:2306.07629.

[28] Matthias Köfferlein. KLayout layout viewer and editor. URL: https://www.klayout.de/.

[29] Manjunath Kudlur, Evan King, James Wang, and Pete Warden. Moonshine v2: Ergodic streaming encoder ASR for latency-critical speech applications, 2026. URL: https://arxiv.org/abs/2602.12241, arXiv:2602.12241.

[30] LibreLane contributors. LibreLane: successor to OpenLane. GitHub repository. URL: https://github.com/librelane/librelane.

[31] Sean Lie. Cerebras architecture deep dive: First look inside the hardware/software co-design for deep learning. *IEEE Micro*, 43(3):18–30, 2023. doi:10.1109/MM.2023.3256384.

[32] Mingjie Liu, Teodor-Dumitru Ene, Robert Kirby, Chris Cheng, Nathaniel Pinckney, Rongjian Liang, Jonah Alben, Himyanshu Anand, et al. ChipNeMo: Domain-adapted LLMs for chip design, 2023. URL: https://arxiv.org/abs/2311.00176, arXiv:2311.00176.

[33] Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. VerilogEval: Evaluating large language models for Verilog code generation, 2023. URL: https://arxiv.org/abs/2309.07544, arXiv:2309.07544.

[34] Shang Liu, Wenji Fang, Yao Lu, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. RTLCoder: Outperforming GPT-3.5 in design RTL generation with our open-source dataset and lightweight solution. In *IEEE LLM Aided Design Workshop (LAD)*, 2024. URL: https://arxiv.org/abs/2312.08617, doi:10.1109/LAD62341.2024.10691788.

[35] Yang Liu, Yi Chen, Yongwei Zhao, Yifan Hao, Zifu Zheng, Weihao Kong, Zhangmai Li, Dongchen Jiang, et al. Hardwired-neurons language processing units as general-purpose cognitive substrates, 2025. URL: https://arxiv.org/abs/2508.16151, arXiv:2508.16151.

[36] Yifei Liu, Jicheng Wen, Yang Wang, Shengyu Ye, Li Lyna Zhang, Ting Cao, Cheng Li, and Mao Yang. VPTQ: Extreme low-bit vector post-training quantization for large language models, 2024. URL: https://arxiv.org/abs/2409.17066, arXiv:2409.17066.

[37] Yiyi Lu, Hoi Ian Au, Junyao Zhang, Jingyu Pan, Guanglei Zhou, Yiting Wang, Jingwei Sun, Ang Li, et al. AutoEDA: Enabling EDA flow automation through microservice-based LLM agents, 2025. URL: https://arxiv.org/abs/2508.01012, arXiv:2508.01012.

[38] Tobias Mann. Etched scores $120M for an ASIC built for transformer models. The Register, June 2024. URL: https://www.theregister.com/2024/06/26/etched_asic_ai/.

[39] Muhammad Husnain Mubarik, Dennis D. Weller, Nathaniel Bleier, Matthew Tomei, Jasmin Aghassi-Hagmann, Mehdi B. Tahoori, and Rakesh Kumar. Printed machine learning classifiers. In *Proceedings of the 53rd IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 73–87, 2020. doi:10.1109/MICRO50266.2020.00019.

[40] Mohnish Pai. Ankhdjet: An open-source compiler for mask-programmed ternary compute-in-ROM on an open PDK, 2026. URL: https://arxiv.org/abs/2608.26206, arXiv:2608.26206.

[41] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. Librispeech: An ASR corpus based on public domain audio books. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5206–5210, 2015. doi:10.1109/ICASSP.2015.7178964.

[42] Gunho Park, Baeseong Park, Minsub Kim, Sungjae Lee, Jeonghoon Kim, Beomseok Kwon, Se Jung Kwon, Byeongwook Kim, et al. LUT-GEMM: Quantized matrix multiplication based on LUTs for efficient inference in large-scale generative language models, 2022. URL: https://arxiv.org/abs/2206.09557, arXiv:2206.09557.

[43] Jongsun Park, Woopyo Jeong, Hunsoo Choo, Hamid Mahmoodi-Meimand, Yongtao Wang, and Kaushik Roy. High performance and low power FIR filter design based on sharing multiplication. In *Proceedings of the International Symposium on Low Power Electronics and Design (ISLPED)*, pages 295–300, 2002. URL: https://www.cecs.uci.edu/~papers/compendium94-03/papers/2002/islped02/pdffiles/11_3.pdf, doi:10.1109/LPE.2002.1029624.

[44] Michael Price, James Glass, and Anantha P. Chandrakasan. A 6 mW, 5,000-word real-time speech recognizer using WFST models. *IEEE Journal of Solid-State Circuits*, 50(1):102–112, 2015. doi:10.1109/JSSC.2014.2367818.

[45] Michael Price, James Glass, and Anantha P. Chandrakasan. A low-power speech recognizer and voice activity detector using deep neural networks. *IEEE Journal of Solid-State Circuits*, 53(1):66–75, 2018. URL: https://people.csail.mit.edu/jrg/2018/Price-JSSC-18.pdf, doi:10.1109/JSSC.2017.2752838.

[46] Qwen Team. Qwen2.5 technical report, 2024. URL: https://arxiv.org/abs/2412.15115, arXiv:2412.15115.

[47] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision, 2022. URL: https://arxiv.org/abs/2212.04356, arXiv:2212.04356.

[48] Mohamed Shalan and Tim Edwards. Building OpenLANE: A 130nm OpenROAD-based tapeout-proven flow. In *Proceedings of the 39th International Conference on Computer-Aided Design (ICCAD)*, pages 1–6, 2020. doi:10.1145/3400302.3415735.

[49] Weiwei Shan, Minhao Yang, Jiaming Xu, Yicheng Lu, Shuai Zhang, Tao Wang, Jun Yang, Longxing Shi, and Mingoo Seok. A 510nW 0.41V low-memory low-computation keyword-spotting chip using serial FFT-based MFCC and binarized depthwise separable convolutional neural network in 28nm CMOS. In *IEEE International Solid-State Circuits Conference (ISSCC)*, pages 230–232, 2020. doi:10.1109/ISSCC19947.2020.9063000.

[50] Wilson Snyder et al. Verilator. Veripool. Accessed 2026-10-24. URL: https://www.veripool.org/verilator/.

[51] STMicroelectronics. STM32N6: Our very own NPU in the most powerful STM32. ST blog, 2024. URL: https://blog.st.com/stm32n6/.

[52] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel Emer. Efficient processing of deep neural networks: A tutorial and survey, 2017. URL: https://arxiv.org/abs/1703.09039, arXiv:1703.09039.

[53] Thierry Tambe, En-Yu Yang, Glenn G. Ko, Yuji Chai, Coleman Hooper, Marco Donato, et al. A 25mm^2 SoC for IoT devices with 18ms noise-robust speech-to-text latency via Bayesian speech denoising and attention-based sequence-to-sequence DNN speech recognition in 16nm FinFET. In *IEEE International Solid-State Circuits Conference (ISSCC)*, pages 158–160, 2021. doi:10.1109/ISSCC42613.2021.9366062.

[54] Thierry Tambe, Jeff Zhang, Coleman Hooper, Tianyu Jia, Paul N. Whatmough, Joseph Zuckerman, et al. A 12nm 18.1TFLOPs/W sparse transformer processor with entropy-based early exit, mixed-precision predication and fine-grained power management. In *IEEE International Solid-State Circuits Conference (ISSCC)*, pages 342–344, 2023. doi:10.1109/ISSCC42615.2023.10067817.

[55] Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks, 2024. URL: https://arxiv.org/abs/2402.04396, arXiv:2402.04396.

[56] Fengbin Tu, Zihan Wu, Yiqi Wang, Ling Liang, Liu Liu, Yufei Ding, et al. A 28nm 15.59μJ/token full-digital bitline-transpose CIM-based sparse transformer accelerator with pipeline/parallel reconfigurable modes. In *IEEE International Solid-State Circuits Conference (ISSCC)*, pages 466–468, 2022. doi:10.1109/ISSCC42614.2022.9731645.

[57] Mart van Baalen, Andrey Kuzmin, Ivan Koryakovskiy, Markus Nagel, Peter Couperus, Cedric Bastoul, Eric Mahurin, Tijmen Blankevoort, et al. GPTVQ: The blessing of dimensionality for LLM quantization, 2024. URL: https://arxiv.org/abs/2402.15319, arXiv:2402.15319.

[58] Matt Venn. Tiny tapeout: A shared silicon tape out platform accessible to everyone. *IEEE Solid-State Circuits Magazine*, 16(2):20–29, 2024. doi:10.1109/MSSC.2024.3381097.

[59] Yevgen Voronenko and Markus Püschel. Multiplierless multiple constant multiplication. *ACM Transactions on Algorithms*, 3(2):11, 2007. doi:10.1145/1240233.1240234.

[60] wafer.space. wafer.space: Budget silicon manufacturing. URL: https://wafer.space.

[61] Wenqiang Wang, Yijia Zhang, Zikai Zhang, Guanting Huo, Hao Liang, Shijie Cao, and Ningyi Xu. ROMA: a read-only-memory-based accelerator for QLoRA-based on-device LLM, 2025. URL: https://arxiv.org/abs/2503.12988, arXiv:2503.12988.

[62] Yang Wang, Yubin Qin, Dazheng Deng, Jingchuan Wei, Yang Zhou, Yuanqi Fan, et al. A 28nm 27.5TOPS/W approximate-computing-based transformer processor with asymptotic sparsity speculating and out-of-order computing. In *IEEE International Solid-State Circuits Conference (ISSCC)*, 2022. doi:10.1109/ISSCC42614.2022.9731686.

[63] Yiting Wang, Wanghao Ye, Yexiao He, Yiran Chen, Gang Qu, and Ang Li. MCP4EDA: LLM-powered model context protocol RTL-to-GDSII automation with backend aware synthesis optimization, 2025. URL: https://arxiv.org/abs/2507.19570, arXiv:2507.19570.

[64] Jianyu Wei, Shijie Cao, Ting Cao, Lingxiao Ma, Lei Wang, Yanyong Zhang, and Mao Yang. T-MAC: CPU renaissance via table lookup for low-bit LLM deployment on edge. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, 2025. URL: https://arxiv.org/abs/2407.00088, doi:10.1145/3689031.3696099.

[65] Stanley A. White. Applications of distributed arithmetic to digital signal processing: A tutorial review. *IEEE ASSP Magazine*, 6(3):4–19, 1989. doi:10.1109/53.29648.

[66] Claire Wolf et al. Yosys open SYnthesis suite. YosysHQ. Accessed 2026-10-24. URL: https://yosyshq.net/yosys/.

[67] Ali Hadi Zadeh, Mostafa Mahmoud, Ameer Abdelhadi, and Andreas Moshovos. Mokey: Enabling narrow fixed-point inference for out-of-the-box floating-point transformer models. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)*, 2022. URL: https://arxiv.org/abs/2203.12758, doi:10.1145/3470496.3527438.

[68] Volker Zota. AI inference cast in silicon: Taalas announces HC1 chip. heise online, February 2026. URL: https://heise.de/-11185112.

A Artifacts

All paths are relative to the RouteVoice repository root, https://github.com/sanjuhs/routevoice. `FIN` denotes `routing-voice-chip-design/overnight-20261014/finish-1023`.

- **Release package (demo clips, simulation logs, layout renders, signoff table, scaling data):** `routing-voice-chip-design/release-v1/`.

- **Bit-exact recognizer with index+codebook weights:** FIN/w11-taalas-e2e/RESULTS.md, results.json.
- **K64 accuracy:** routing-voice-chip-design/docs/TAALAS-K64-QUALITY-RESULT.md.
- **Engine contract and bench:** routing-voice-chip-design/overnight-20261014/fabric-k64/CONTRACT.md, RESULTS.md.
- **Engine routing history:**
  - first route: FIN/w10-route/RESULTS.md;
  - closure passes: FIN/w10-route/closure-lead/;
  - first-route power: FIN/w10-route/routed-power-lead/.
- **Final engine (fix 2):** FIN/w10-route/pipe2/ (README, generator make_engine.py, bench, flow configuration, evidence/; antenna close-out in evidence-ant/).
- **Final engine run backup (fix 2 and antenna close-out, checksum manifests):** Hugging Face dataset sanjuhs/routevoice-engine-fix2-20261024.
- **Full-recognizer synthesis:** FIN/w12-synth/RESULTS.md.
- **Weight banks:** FIN/w13-arrays/RESULTS.md, DRIVERS-RESULTS.md.
- **Logic macros and two-die integration:** FIN/w14-blocks/, FIN/w15-chip/.
- **Chronological lab log:** OVERNIGHT-PROGRESS.md.
- **Cloud run backup (53 GB, checksum manifest):** Hugging Face dataset sanjuhs/routevoice-pod-final-20261022.